Interop Resolution Service

API reference for Onyx: member resolution, `POST /v1/resolve`.

Version v1 · Point32Health

---

1 Overview

Member resolution for Onyx, POST /v1/resolve: resolves an EMR-supplied member id against the plan's member records, reports whether coverage is active on the date of service, and returns the id as resolved and in every UM vendor's format, each with the payer for that vendor's request. Every answer travels in an envelope: data and header.

One call per prior authorization. Send the member id exactly as the provider's EMR supplied it, the date of service (or the period of service) and, when known, the patient's date of birth. The answer says whether the member was found, whether coverage is active on that date, the line of business to route on, and the member id in every UM vendor's format with the payer to put on that vendor's request.

> **Base URL and access.** The base URL is issued by Point32Health for each environment and is not part of this document; the path below is relative to it. Authentication is OAuth 2.0: work with Point32Health to obtain the client credentials and the token endpoint for each environment, and send the access token on every call in the `Authorization` header. Nothing in the body or the path carries a credential. A rejected or expired token is answered in front of the service (401 or 403) without the envelope of section 2.

---

2 The operation and the envelope

`POST /v1/resolve` with `Content-Type: application/json`. The service always answers JSON, whatever `Accept` header is sent.

Finds the member behind the id the EMR typed, says whether coverage is active on the date of service (or on every day of a period of service), and returns the id as resolved and in every configured UM vendor's format (memberId.forVendors), each entry with the payer for that vendor's request (payerId, payerName). When several members on the plan match and no dateOfBirth settles it, the outcome is AMBIGUOUS and the message says what to resend; the other members are never listed. clientId, clientType (EXT), requestId and memberId are required; unknown properties are ignored. A sent date is the date evaluated; an unusable date is a 400. POST because the member id is PHI and must not appear in URLs; the operation is a pure read and may be repeated.

The envelope

Every answer, success or error, is one JSON object with two members: `data` and `header`. On a success, `data` is the member resolution answer of section 4. On an error, `data` holds the error details of section 6. `header` is the same on both:

Field	Type	Required	Meaning
`header.httpStatus`	integer	yes	The HTTP status of the answer, the same value as on the wire

Field	Type	Required	Meaning
`header.message`	string	yes	A short text about the answer; on an error it may repeat the code
`header.status`	success | error	yes	success or error
`header.statusCode`	string		On an error only: the error code to branch on; the error catalog in section 6 lists them
`header.timestamp`	integer (epoch ms)	yes	When the answer was produced, milliseconds since 1970-01-01 UTC

The HTTP status on the wire and `header.httpStatus` always agree; branch on either. On an error, `header.statusCode` is the error code to branch on and `header.status` is `error`.

Headers

Header	Direction	Rule
`X-Correlation-Id`	in	Optional transport-level id, 1 to 64 characters of letters, digits, `.`, `_`, `:` or `-`; opaque. Missing or invalid: the service generates one. The caller's own id for the call is `requestId` in the body.
`X-Correlation-Id`	out	The echoed or generated value, for support.
`Retry-After`	out	On 503, when present: seconds to wait before retrying; 10 when the header is absent.

Responses

Status	Meaning
`200`	outcome ACTIVE | INACTIVE | NOT_FOUND | AMBIGUOUS; memberId.forVendors, each vendor with its payer, present for ACTIVE and INACTIVE
`400`	INVALID_REQUEST for a missing or unusable clientId, clientType or requestId, a missing memberId, an unusable date or a body that is not JSON; MEMBER_LOOKUP_REJECTED when the member lookup answered 400 (data.details[0].code HTTP_400, the lookup's text in the message)
`422`	DOB_MISMATCH: the date of birth matches no record for this id
`404`	INVALID_REQUEST / ROUTE_NOT_FOUND: wrong path; a connector defect (a member that is not found is a 200 with outcome NOT_FOUND)
`405`	INVALID_REQUEST / METHOD_NOT_ALLOWED: the operation is POST only
`415`	INVALID_REQUEST / UNSUPPORTED_MEDIA_TYPE: the request body must be application/json
`502`	MEMBER_LOOKUP_ERROR or MEMBER_LOOKUP_INVALID_RESPONSE (the member lookup); MEMBER_PLAN_ERROR or MEMBER_PLAN_INVALID_RESPONSE (the member plan lookup)
`503`	MEMBER_LOOKUP_UNAVAILABLE or MEMBER_PLAN_UNAVAILABLE; retry after the Retry-After seconds; header `Retry-After` (Seconds to wait before retrying; 10 when absent)
`500`	INTERNAL_ERROR: the service itself failed; retry once later, then alert the service owners with the requestId and the X-Correlation-Id response header

3 Request

Member resolution request: clientId, clientType, requestId and memberId are required; unknown properties are ignored; a sent date must be usable. Unknown properties are ignored, so a `vendor` sent by habit does no harm: the answer is always for every vendor.

Field	Type	Required	Example	Meaning
`clientId`	string	yes	`ONYX`	The calling system, as registered with the plan; Onyx sends ONYX. A plain name in the body, not the OAuth 2.0 client id. Letters, digits, '.', '_' or '-', at most 50 characters
`clientType`	EXT	yes	`EXT`	Always EXT: Onyx is a caller outside the plan
`requestId`	string	yes	`3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13`	The caller's id for this call, unique per request; echoed in the response and in every error response. Letters, digits, '.', '_', ':' or '-', at most 64 characters
`memberId`	string	yes	`S########`	The member id exactly as the EMR supplied it. Looked up unchanged (surrounding whitespace removed); the service never checks its shape, length or characters. For instance a THP Medicare card number is a letter and 8 digits; the example is that pattern, masked
`dateOfService`	string		`2026-10-15`	Date of service, yyyy-MM-dd; the first date when the service covers a period. Omitted, null or blank: today in US Eastern time (America/New_York) is used, dateOfServiceDefaulted is true and the answer's dateOfService says which date was used; send the date whenever you have it. A sent value is the date evaluated: any future date is judged on the coverage on record; not a real date (DATE_OF_SERVICE_INVALID), or more than 10 years back (DATE_OF_SERVICE_OUT_OF_RANGE), is a 400.
`dateOfServiceEnd`	string		`2026-10-20`	Optional last date of service, yyyy-MM-dd, when the service covers a period: coverage must hold on every day from dateOfService to it. Not before dateOfService; needs dateOfService. Omitted, null or blank counts as not sent.
`dateOfBirth`	string		`1950-03-15`	Optional patient date of birth, yyyy-MM-dd, used only to verify the member or to tell apart the members on a plan who share the id. Omitted, null or blank counts as not sent. Not a real date is a 400 (DATE_OF_BIRTH_INVALID); in the future or more than 125 years ago is a 400 (DATE_OF_BIRTH_OUT_OF_RANGE). Never logged, never echoed.

4 Response (200)

The answer: the outcome Onyx branches on, one sentence for a person, the member id as received, as resolved and in every vendor's format with that vendor's payer, the line of business, the date evaluated, the coverage flag with its period, the caller's requestId and a traceId for support. A field that does not apply is absent, never null. The answer is the `data` member of the envelope; fields are sent in the order listed. Lists of values marked "for example" are not closed: new vendors and new lines of business can be added, so never reject a value that is not in this document.

Field	Type	Required	Meaning
`data`	object	yes	The answer: the outcome Onyx branches on, one sentence for a person, the member id as received, as resolved and in every vendor's format with that vendor's payer, the line of business, the date evaluated, the coverage flag with its period, the caller's requestId and a traceId for support. A field that does not apply is absent, never null
`data.outcome`	ACTIVE | INACTIVE | NOT_FOUND | AMBIGUOUS	yes	ACTIVE: member found, coverage active on the date of service (or on every day of the period). INACTIVE: member found, not covered on that date or not on every day of the period; message says why. NOT_FOUND: no member for this id on that date. AMBIGUOUS: several members match and nothing sent tells them apart; message says what to resend
`data.message`	string	yes	One sentence for a person reading the answer. Branch on outcome, not on this text
`data.memberId`	object	yes	The member id as received, as resolved, and in every vendor's format
`data.memberId.received`	string	yes	The member id as sent, surrounding whitespace removed
`data.memberId.resolved`	string		The id the lookup resolved to, as the plan stores it; for a converted member a different number from the one received. Present on ACTIVE and INACTIVE answers only
`data.memberId.forVendors[]`	array		The resolved id in every configured vendor's format, each with the payer for that vendor's request; one entry per vendor, sorted by vendor code. Present on ACTIVE and INACTIVE answers only
`data.memberId.forVendors[].vendor`	string		The vendor code, for example EVICORE; today CARELON, EVICORE, EVOLENT, MHK, ONYX and OPTUM. Vendors can be added, so never treat the list as closed
`data.memberId.forVendors[].memberId`	string		The resolved id in this vendor's format: the member id to put on this vendor's request
`data.memberId.forVendors[].payerId`	string		The payer id to put on this vendor's request; it differs by vendor and by the member's heritage company and plan population, so use the entry's value as given
`data.memberId.forVendors[].payerName`	string		The payer name that goes with payerId
`data.memberId.forVendors[].name[]`	array		The member's name as the plan has it on record, one entry in the FHIR HumanName shape: given[0] the first name, given[1] the middle name when there is one, family the last name, text all of them joined. Put it on the vendor's request as given. Absent only when the plan has no name on record; then use the name from the EMR
`data.memberId.forVendors[].name[].text`	string		The full name: the given names and the family name joined with spaces
`data.memberId.forVendors[].name[].family`	string		The last name; absent when the plan has none on record
`data.memberId.forVendors[].name[].given`	array		The first name, then the middle name when there is one; absent when the plan has no first name on record

Field	Type	Required	Meaning
`data.lineOfBusiness`	string		The line of business Onyx routes on, from the plan in effect on the date of service, for example SCO, MA-HMO, MA-PPO, D-SNP, MA-TOGETHER, RI-TOGETHER or MA-QHP; values can be added, so never treat the list as closed. When no rule applies, the member record's own category is passed through (for example MCR, PP, COM), and the field is absent when the record carries none. Present on ACTIVE and INACTIVE answers only. Agree with Point32Health how to route a value outside the list, or an absent one
`data.dateOfService`	date (yyyy-MM-dd)	yes	The date evaluated; the first date when a period was asked about
`data.dateOfServiceEnd`	date (yyyy-MM-dd)		The last date evaluated; present only when the request asked about a period
`data.dateOfServiceDefaulted`	boolean	yes	true when no dateOfService was sent and today was used
`data.coverage`	object		The coverage flag and the coverage period behind it. coverageId, effectiveDate and endDate are present when active, and when inactive only because the period ends before the last date asked about; otherwise only active is sent and message says why. Present on ACTIVE and INACTIVE answers only
`data.coverage.coverageId`	string		An id for the coverage period: the resolved id reduced to letters and digits, then the effective and end dates as yyyyMMdd, run together with no separator (39991231 for an open-ended period). Present whenever the dates are; the same member with the same coverage always gets the same value
`data.coverage.active`	boolean	yes	The flag: true when coverage holds on the date of service, or on every day of the period of service
`data.coverage.effectiveDate`	date (yyyy-MM-dd)		First day of the coverage period
`data.coverage.endDate`	date (yyyy-MM-dd)		Last day of the coverage period; 3999-12-31 for an open-ended period
`data.requestId`	string	yes	The caller's requestId, echoed
`data.traceId`	string	yes	The id of the member lookup behind this answer; quote it to support
`data.sourceMessage`	object		What the member lookup said when it had no member for the id. NOT_FOUND only, and only when the lookup sent a message
`data.sourceMessage.type`	string		The message type as the lookup sent it
`data.sourceMessage.status`	string		The status as the lookup sent it
`data.sourceMessage.code`	string		The message code as the lookup sent it
`data.sourceMessage.text`	string		The message text as the lookup sent it
`header`	object	yes	The envelope header on every answer

Field	Type	Required	Meaning
`header.httpStatus`	integer	yes	The HTTP status of the answer, the same value as on the wire
`header.message`	string	yes	A short text about the answer; on an error it may repeat the code
`header.status`	success | error	yes	success or error
`header.statusCode`	string		On an error only: the error code to branch on; the error catalog in section 6 lists them
`header.timestamp`	integer (epoch ms)	yes	When the answer was produced, milliseconds since 1970-01-01 UTC

An AMBIGUOUS answer never lists the members who share the id.

---

5 What to do with each answer

Onyx branches on `data.outcome` for a 200, and on the HTTP status (or `header.httpStatus`) otherwise. Two rules for every row: Onyx never reformats a member id itself when the service is up, and Onyx keeps the full response with its transaction.

Signal	Meaning	Onyx action
`200` `ACTIVE`	Member found; coverage active on the date of service, or on every day of the period of service (`data.coverage.active` is true).	Send each vendor its `data.memberId.forVendors` entry: the `memberId` of the entry whose `vendor` is that vendor, already in the vendor's format, with that entry's `payerId`, `payerName` and `name` (the member's name as the plan has it, one FHIR HumanName) on the request. Route on `data.lineOfBusiness`. Keep `data.coverage.coverageId` with the transaction as the reference to the coverage period behind the decision.
`200` `INACTIVE`	Member found; not covered on that date, or not on every day of the period. `data.message` says why.	Hold for intake. `data.memberId.forVendors` and `data.lineOfBusiness` are still there if the business rule says to submit anyway.
`200` `NOT_FOUND`	No member for this id on that date. `data.sourceMessage` carries what the member lookup said, when it sent a message.	"Member not found" worklist. If Onyx holds a fuller form of the id (for instance with the suffix), one resend with it.
`200` `AMBIGUOUS`	Several members on the plan share the id and nothing sent tells them apart. Nobody is listed.	One resend with `dateOfBirth` or the member's full id including the suffix. If it is still AMBIGUOUS (twins share a date of birth, or the records carry none), intake takes the full id from the member's card and resends.
`400` / `422`	Request or identity problem (the error catalog below).	Never retry. Route by `header.statusCode` and `data.details[].code`.
`502` / `503` / `500`	A lookup behind the service, or the service itself, could not answer.	Retry as section 8 says, always with a new `requestId`; when the retries are used up, park the transaction and alert the service owners.

Signal	Meaning	Onyx action
`401` / `403` / `429`	Answered by the API gateway in front of the service, without the envelope of section 2.	401 and 403: a token problem; obtain a fresh token, never retry the same one. 429: treat like a 503 and retry after its `Retry-After`.
No response at all (connection refused, timeout)	The service is down or unreachable.	Retry once with a new `requestId`, as section 8 says; still no response: pass the member id to the UM vendor exactly as the EMR supplied it, and record the `requestId` so the transaction can be reconciled later.

---

6 Errors

Every non-200 answer produced by the service is the envelope of section 2 with `header.status` `error`, the error code in `header.statusCode` and the details in `data.details`. Answers produced by the API gateway in front of it (401, 403, 429, or any response whose body is not JSON) do not carry the envelope: treat 401 and 403 as a token problem and never retry the same token; treat 429 like a 503 and retry after its `Retry-After`. `header.statusCode` is stable and is what to branch on; `data.details[].code` refines it; `data.requestId` echoes the caller's id whenever it was usable, so a 400 about another field still carries it, and is an empty string otherwise; `data.traceId` is present once the member lookup was made.

Field	Type	Required	Meaning
`data`	object	yes	The data part of an error answer
`data.requestId`	string	yes	The caller's requestId when the request carried a usable one, else an empty string
`data.traceId`	string		The id of the member lookup behind the answer, present once the lookup was made
`data.details[]`	array	yes	One entry per problem found; every problem with a request is reported together. Empty on a 500
`data.details[].code`	string	yes	The detail code that refines header.statusCode, for example MEMBER_ID_MISSING, DATE_OF_SERVICE_INVALID, HTTP_400, READ_TIMEOUT
`data.details[].field`	string		The request field the problem is about, when there is one
`data.details[].message`	string	yes	One sentence for a person
`header`	object	yes	The envelope header on every answer
`header.httpStatus`	integer	yes	The HTTP status of the answer, the same value as on the wire
`header.message`	string	yes	A short text about the answer; on an error it may repeat the code
`header.status`	success | error	yes	success or error
`header.statusCode`	string		On an error only: the error code to branch on; the error catalog in section 6 lists them

Field	Type	Required	Meaning
`header.timestamp`	integer (epoch ms)	yes	When the answer was produced, milliseconds since 1970-01-01 UTC

Error catalog

HTTP	`header.statusCode`	`data.details[].code`	Meaning and Onyx action

`400`	`INVALID_REQUEST`	`CLIENT_ID_MISSING`
		`CLIENT_ID_INVALID`
		`CLIENT_TYPE_MISSING`
		`CLIENT_TYPE_INVALID`
		`REQUEST_ID_MISSING`
		`REQUEST_ID_INVALID`
		`MEMBER_ID_MISSING`
		`DATE_OF_SERVICE_INVALID`
		`DATE_OF_SERVICE_OUT_OF_RANGE`
		`DATE_OF_SERVICE_END_INVALID`
		`DATE_OF_SERVICE_END_BEFORE_START`
		`DATE_OF_SERVICE_END_WITHOUT_START`
		`DATE_OF_BIRTH_INVALID`
		`DATE_OF_BIRTH_OUT_OF_RANGE`
		`MALFORMED_JSON`
		`WRONG_JSON_TYPE`
			Never retry. Every problem with the request is reported together, one detail per problem. MEMBER_ID_MISSING, DATE_OF_SERVICE_OUT_OF_RANGE and DATE_OF_BIRTH_OUT_OF_RANGE are provider data; the caller codes are configuration; the rest are mapping defects.

`400`	`MEMBER_LOOKUP_REJECTED`	`HTTP_400`	The member lookup could not process the request as sent; the detail message carries its text. Never retry; send the request and the message to the service owners.
`422`	`DOB_MISMATCH`	`DOB_MISMATCH`	The date of birth matches no record for this id. Manual identity review; never file the authorization.

`404`, `405`, `415`	`INVALID_REQUEST`	`ROUTE_NOT_FOUND`
		`METHOD_NOT_ALLOWED`
		`UNSUPPORTED_MEDIA_TYPE`
			Wrong path, method or request content type: a connector defect. ROUTE_NOT_FOUND is the only 404 this service returns; a member that is not found is a 200 with outcome NOT_FOUND.

`502`	`MEMBER_LOOKUP_ERROR`
	`MEMBER_LOOKUP_INVALID_RESPONSE`
		`HTTP_<code>`
		`ERROR_MESSAGE`
		`UNPARSEABLE_BODY`
		`EMPTY_BODY`
		`NO_MEMBER_ID`
		`UNREADABLE_COVERAGE`
		`CLIENT_ERROR`
			The member lookup answered outside its contract, or with data the service could not read. Retry once with a new requestId; if it fails again, park the transaction and alert the service owners.

`502`	`MEMBER_PLAN_ERROR`
	`MEMBER_PLAN_INVALID_RESPONSE`
		`HTTP_<code>`
		`UNPARSEABLE_BODY`
		`EMPTY_BODY`
		`CLIENT_ERROR`
			The member plan lookup answered outside its contract, or with data the service could not read. Retry once with a new requestId; if it fails again, park the transaction and alert the service owners.

HTTP	`header.statusCode`	`data.details[].code`	Meaning and Onyx action

`503`	`MEMBER_LOOKUP_UNAVAILABLE`
	`MEMBER_PLAN_UNAVAILABLE`
		`CONNECT_FAILED`
		`READ_TIMEOUT`
		`HTTP_500`
		`HTTP_5xx`
		`HTTP_429`
		`HTTP_408`
			A lookup could not be reached, timed out or is temporarily unavailable. Retry after the Retry-After seconds (10 when the header is absent) with a new requestId, as section 8 says; when the retries are used up, park the transaction and alert the service owners.

`500`	`INTERNAL_ERROR`	(none: data.details is empty)	The service itself failed. Retry once with a new requestId; if it fails again, park the transaction and alert the service owners with the requestId and the X-Correlation-Id.

---

7 Examples

Answers of the service against test data, in the envelope of section 2; the member ids and dates are fictitious. Where a message or `sourceMessage` forwards the lookup's own code and text, the example shows a placeholder, because the live values differ: branch on `header.statusCode` and `data.details[].code` only.

Request

```
POST /v1/resolve
Content-Type: application/json
X-Correlation-Id: ONYX-PA-2026-000123

{
  "clientId": "ONYX",
  "clientType": "EXT",
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "memberId": "123456789",
  "dateOfService": "2026-10-15",
  "dateOfBirth": "1950-03-15"
}
```

`200` ACTIVE, THP Medicare card number (a 9-character core) with the date of birth verified: the id in every vendor's format

```
{
  "data": {
    "outcome": "ACTIVE",
    "message": "Member found; coverage active on 2026-10-15",
    "memberId": {
      "received": "123456789",
      "resolved": "123456789   01",
      "forVendors": [
        {
          "vendor": "CARELON",
          "memberId": "12345678901",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Taylor A Morgan", "family": "Morgan", "given": ["Taylor", "A"]}
          ]
```

```
      },
      {
        "vendor": "EVICORE",
        "memberId": "12345678901",
        "payerId": "TUFTS",
        "payerName": "TUFTS",
        "name": [
          {"text": "Taylor A Morgan", "family": "Morgan", "given": ["Taylor", "A"]}
        ]
      },
      {
        "vendor": "EVOLENT",
        "memberId": "12345678901",
        "payerId": "Tufts Health Plan",
        "payerName": "Tufts Health Plan",
        "name": [
          {"text": "Taylor A Morgan", "family": "Morgan", "given": ["Taylor", "A"]}
        ]
      },
      {
        "vendor": "MHK",
        "memberId": "123456789   01",
        "payerId": "Point32Health",
        "payerName": "Point32Health",
        "name": [
          {"text": "Taylor A Morgan", "family": "Morgan", "given": ["Taylor", "A"]}
        ]
      },
      {
        "vendor": "ONYX",
        "memberId": "12345678901",
        "payerId": "Point32Health",
        "payerName": "Point32Health",
        "name": [
          {"text": "Taylor A Morgan", "family": "Morgan", "given": ["Taylor", "A"]}
        ]
      },
      {
        "vendor": "OPTUM",
        "memberId": "123456789",
        "payerId": "Point32Health",
        "payerName": "Point32Health",
        "name": [
          {"text": "Taylor A Morgan", "family": "Morgan", "given": ["Taylor", "A"]}
        ]
      }
    ]
  },
  "lineOfBusiness": "MCR",
  "dateOfService": "2026-10-15",
  "dateOfServiceDefaulted": false,
  "coverage": {
    "coverageId": "12345678901202101010139991231",
    "active": true,
    "effectiveDate": "2021-01-01",
    "endDate": "3999-12-31"
  },
  "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
  "traceId": "INTEROP-1791253852491-77574"
},
"header": {
  "httpStatus": 200,
  "message": "Request processed successfully",
```

```
    "status": "success",
    "timestamp": 1791577774968
  }
}
```

The requests below carry the same `clientId`, `clientType` and `requestId`; only the member fields are shown.

`200` INACTIVE, coverage ended: the ids are still returned

```
{"memberId": "234567890", "dateOfService": "2026-10-15"}

{
  "data": {
    "outcome": "INACTIVE",
    "message": "Member found; coverage ended before 2026-10-15",
    "memberId": {
      "received": "234567890",
      "resolved": "234567890   01",
      "forVendors": [
        {
          "vendor": "CARELON",
          "memberId": "23456789001",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Jordan Lee", "family": "Lee", "given": ["Jordan"]}
          ]
        },
        {
          "vendor": "EVICORE",
          "memberId": "23456789001",
          "payerId": "TUFTS",
          "payerName": "TUFTS",
          "name": [
            {"text": "Jordan Lee", "family": "Lee", "given": ["Jordan"]}
          ]
        },
        {
          "vendor": "EVOLENT",
          "memberId": "23456789001",
          "payerId": "Tufts Health Plan",
          "payerName": "Tufts Health Plan",
          "name": [
            {"text": "Jordan Lee", "family": "Lee", "given": ["Jordan"]}
          ]
        },
        {
          "vendor": "MHK",
          "memberId": "234567890   01",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Jordan Lee", "family": "Lee", "given": ["Jordan"]}
          ]
        },
        {
          "vendor": "ONYX",
          "memberId": "23456789001",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
```

```
            {"text": "Jordan Lee", "family": "Lee", "given": ["Jordan"]}
          ]
        },
        {
          "vendor": "OPTUM",
          "memberId": "234567890",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Jordan Lee", "family": "Lee", "given": ["Jordan"]}
          ]
        }
      ]
    },
    "lineOfBusiness": "MCR",
    "dateOfService": "2026-10-15",
    "dateOfServiceDefaulted": false,
    "coverage": {"active": false},
    "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
    "traceId": "INTEROP-1791253852504-23697"
  },
  "header": {
    "httpStatus": 200,
    "message": "Request processed successfully",
    "status": "success",
    "timestamp": 1791577776968
  }
}
```


No plan is in effect on that date, so `lineOfBusiness` falls back to the member record's category.

`200` AMBIGUOUS, 9-character Together family, no date of birth


```
{"memberId": "345678901", "dateOfService": "2026-10-15"}

{
  "data": {
    "outcome": "AMBIGUOUS",
    "message": "Several members match this id; resend with dateOfBirth or the member's full id including the suffix",
    "memberId": {"received": "345678901"},
    "dateOfService": "2026-10-15",
    "dateOfServiceDefaulted": false,
    "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
    "traceId": "INTEROP-1791255194024-32990"
  },
  "header": {
    "httpStatus": 200,
    "message": "Request processed successfully",
    "status": "success",
    "timestamp": 1791577778968
  }
}
```


`200` NOT_FOUND, no date of service sent: the member lookup answered 404, forwarded in `sourceMessage`


```
{"memberId": "HP111222333"}

{
  "data": {
    "outcome": "NOT_FOUND",
```

```
    "message": "No member found for this id",
    "memberId": {"received": "HP111222333"},
    "dateOfService": "2026-10-05",
    "dateOfServiceDefaulted": true,
    "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
    "traceId": "INTEROP-1791253852517-51447",
    "sourceMessage": {
      "type": "<as the lookup sent it>",
      "status": "<as the lookup sent it>",
      "code": "<as the lookup sent it>",
      "text": "<as the lookup sent it>"
    }
  },
  "header": {
    "httpStatus": 200,
    "message": "Request processed successfully",
    "status": "success",
    "timestamp": 1791577779968
  }
}
```


200 ACTIVE, converted member, old THP id, date after migration


```
{"memberId": "567890123 01", "dateOfService": "2026-10-15"}

{
  "data": {
    "outcome": "ACTIVE",
    "message": "Member found; coverage active on 2026-10-15",
    "memberId": {
      "received": "567890123 01",
      "resolved": "HP567890123",
      "forVendors": [
        {
          "vendor": "CARELON",
          "memberId": "HP567890123",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Sam Patel", "family": "Patel", "given": ["Sam"]}
          ]
        },
        {
          "vendor": "EVICORE",
          "memberId": "HP567890123",
          "payerId": "HARVARD PILGRIM HEALTH",
          "payerName": "HARVARD PILGRIM HEALTH",
          "name": [
            {"text": "Sam Patel", "family": "Patel", "given": ["Sam"]}
          ]
        },
        {
          "vendor": "EVOLENT",
          "memberId": "HP567890123",
          "payerId": "Harvard Pilgrim Health Care",
          "payerName": "Harvard Pilgrim Health Care",
          "name": [
            {"text": "Sam Patel", "family": "Patel", "given": ["Sam"]}
          ]
        },
        {
          "vendor": "MHK",
```

```
          "memberId": "HP567890123",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Sam Patel", "family": "Patel", "given": ["Sam"]}
          ]
        },
        {
          "vendor": "ONYX",
          "memberId": "HP567890123",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Sam Patel", "family": "Patel", "given": ["Sam"]}
          ]
        },
        {
          "vendor": "OPTUM",
          "memberId": "HP567890123",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Sam Patel", "family": "Patel", "given": ["Sam"]}
          ]
        }
      ]
    },
    "lineOfBusiness": "COM",
    "dateOfService": "2026-10-15",
    "dateOfServiceDefaulted": false,
    "coverage": {
      "coverageId": "HP567890123202001013999123 1",
      "active": true,
      "effectiveDate": "2020-01-01",
      "endDate": "3999-12-31"
    },
    "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
    "traceId": "INTEROP-1791253852525-35782"
  },
  "header": {
    "httpStatus": 200,
    "message": "Request processed successfully",
    "status": "success",
    "timestamp": 1791577781968
  }
}
```


422 DOB_MISMATCH


```
{
  "data": {
    "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
    "traceId": "INTEROP-1791253852529-13534",
    "details": [
      {
        "code": "DOB_MISMATCH",
        "field": "dateOfBirth",
        "message": "no returned record has this date of birth"
      }
    ]
  },
  "header": {
```

```
    "httpStatus": 422,
    "message": "DOB_MISMATCH",
    "status": "error",
    "statusCode": "DOB_MISMATCH",
    "timestamp": 1791577782968
  }
}
```

`503` MEMBER_LOOKUP_UNAVAILABLE: the member lookup answered 500, its code and text forwarded, `Retry-After: 10`

```
{
  "data": {
    "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
    "traceId": "INTEROP-1791253852533-40365",
    "details": [
      {
        "code": "HTTP_500",
        "message": "The member lookup reported an internal error: <the lookup's code and text>"
      }
    ]
  },
  "header": {
    "httpStatus": 503,
    "message": "MEMBER_LOOKUP_UNAVAILABLE",
    "status": "error",
    "statusCode": "MEMBER_LOOKUP_UNAVAILABLE",
    "timestamp": 1791577783968
  }
}
```

`400` INVALID_REQUEST, two problems at once

```
{"memberId": "", "dateOfService": "10/15/2026"}

{
  "data": {
    "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
    "details": [
      {"code": "MEMBER_ID_MISSING", "field": "memberId", "message": "memberId is required"},
      {
        "code": "DATE_OF_SERVICE_INVALID",
        "field": "dateOfService",
        "message": "expected a real date in yyyy-MM-dd"
      }
    ]
  },
  "header": {
    "httpStatus": 400,
    "message": "INVALID_REQUEST",
    "status": "error",
    "statusCode": "INVALID_REQUEST",
    "timestamp": 1791577786968
  }
}
```

200 ACTIVE for a period of service across two plan-year records


```
{"memberId": "U20252026", "dateOfService": "2025-12-20", "dateOfServiceEnd": "2026-01-05"}

{
  "data": {
    "outcome": "ACTIVE",
    "message": "Member found; coverage active from 2025-12-20 to 2026-01-05",
    "memberId": {
      "received": "U20252026",
      "resolved": "U20252026   01",
      "forVendors": [
        {
          "vendor": "CARELON",
          "memberId": "U2025202601",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Uma Continuous", "family": "Continuous", "given": ["Uma"]}
          ]
        },
        {
          "vendor": "EVICORE",
          "memberId": "U2025202601",
          "payerId": "TUFTS",
          "payerName": "TUFTS",
          "name": [
            {"text": "Uma Continuous", "family": "Continuous", "given": ["Uma"]}
          ]
        },
        {
          "vendor": "EVOLENT",
          "memberId": "U2025202601",
          "payerId": "Tufts Health Plan",
          "payerName": "Tufts Health Plan",
          "name": [
            {"text": "Uma Continuous", "family": "Continuous", "given": ["Uma"]}
          ]
        },
        {
          "vendor": "MHK",
          "memberId": "U20252026   01",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Uma Continuous", "family": "Continuous", "given": ["Uma"]}
          ]
        },
        {
          "vendor": "ONYX",
          "memberId": "U2025202601",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Uma Continuous", "family": "Continuous", "given": ["Uma"]}
          ]
        },
        {
          "vendor": "OPTUM",
          "memberId": "U20252026",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
```

```
            {"text": "Uma Continuous", "family": "Continuous", "given": ["Uma"]}
          ]
        }
      ]
    },
    "lineOfBusiness": "MCR",
    "dateOfService": "2025-12-20",
    "dateOfServiceEnd": "2026-01-05",
    "dateOfServiceDefaulted": false,
    "coverage": {
      "coverageId": "U20252026012025010120261231",
      "active": true,
      "effectiveDate": "2025-01-01",
      "endDate": "2026-12-31"
    },
    "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
    "traceId": "INTEROP-1791253852552-13365"
  },
  "header": {
    "httpStatus": 200,
    "message": "Request processed successfully",
    "status": "success",
    "timestamp": 1791577789968
  }
}
```


This test member has one record for 2025 and one for 2026. They touch, so they are one continuous period, 2025-01-01 to 2026-12-31, and the period of service inside it is ACTIVE.

200 INACTIVE for a period of service that outlasts the coverage


```
{"memberId": "U20252026", "dateOfService": "2026-12-20", "dateOfServiceEnd": "2027-01-05"}

{
  "data": {
    "outcome": "INACTIVE",
    "message": "Member found; coverage active on 2026-12-20 but ends 2026-12-31, before 2027-01-05",
    "memberId": {
      "received": "U20252026",
      "resolved": "U20252026   01",
      "forVendors": [
        {
          "vendor": "CARELON",
          "memberId": "U2025202601",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Uma Continuous", "family": "Continuous", "given": ["Uma"]}
          ]
        },
        {
          "vendor": "EVICORE",
          "memberId": "U2025202601",
          "payerId": "TUFTS",
          "payerName": "TUFTS",
          "name": [
            {"text": "Uma Continuous", "family": "Continuous", "given": ["Uma"]}
          ]
        },
        {
          "vendor": "EVOLENT",
          "memberId": "U2025202601",
```

```
          "payerId": "Tufts Health Plan",
          "payerName": "Tufts Health Plan",
          "name": [
            {"text": "Uma Continuous", "family": "Continuous", "given": ["Uma"]}
          ]
        },
        {
          "vendor": "MHK",
          "memberId": "U20252026   01",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Uma Continuous", "family": "Continuous", "given": ["Uma"]}
          ]
        },
        {
          "vendor": "ONYX",
          "memberId": "U2025202601",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Uma Continuous", "family": "Continuous", "given": ["Uma"]}
          ]
        },
        {
          "vendor": "OPTUM",
          "memberId": "U20252026",
          "payerId": "Point32Health",
          "payerName": "Point32Health",
          "name": [
            {"text": "Uma Continuous", "family": "Continuous", "given": ["Uma"]}
          ]
        }
      ]
    },
    "lineOfBusiness": "MCR",
    "dateOfService": "2026-12-20",
    "dateOfServiceEnd": "2027-01-05",
    "dateOfServiceDefaulted": false,
    "coverage": {
      "coverageId": "U2025202601202501012026123 1",
      "active": false,
      "effectiveDate": "2025-01-01",
      "endDate": "2026-12-31"
    },
    "requestId": "3f6c2a9e-8b1d-4e7a-9c5f-2d4b6a8e0c13",
    "traceId": "INTEROP-1791253852556-60202"
  },
  "header": {
    "httpStatus": 200,
    "message": "Request processed successfully",
    "status": "success",
    "timestamp": 1791577790968
  }
}
```


Covered on the first day, not on the last: the flag is false, the message says where coverage ends, and the period is still shown so intake sees how far it goes.

8 Timeouts, retries, security

- **Timeouts.** A successful answer normally takes well under a second. When a lookup behind the service cannot answer, the failure is reported within about 15 seconds; set the client timeout above that.
- **Retry policy.** 503: wait the `Retry-After` seconds (10 when the header is absent) and retry; the number of attempts is to be agreed with Point32Health (three until then). 502 and 500: retry once. 429 from the gateway: treat like a 503. No response at all (connection refused, timeout): retry once. Never retry a 400 or 422, and never retry a 401 or 403 with the same token. Every retry carries a new `requestId`. When the retries are used up: for 502, 503 and 500, park the transaction and alert the service owners with the `requestId` and the `X-Correlation-Id`; for no response at all, pass the member id to the UM vendor exactly as the EMR supplied it and record the `requestId` so the transaction can be reconciled later.
- **Security.** OAuth 2.0 access tokens, issued per environment by Point32Health, on every call; certificates and rate limits are handled by the API gateway in front of the service. The member id, the member's name and the date of birth are PHI: send them in the body only, never in a URL, and do not log the date of birth.
- **Caller identification.** Onyx sends `clientId` `ONYX` and `clientType` `EXT` on every call, and a new `requestId` per call; support finds a call by that `requestId` or by the `traceId` in the answer.